

Minecraft Map: Data Analysis Methods

1 Node/Community Attributes

1.1 Hidden Influence

Hidden Influence is a metric designed to give high scores to channels that “punch above their weight”, or, in other words, have relatively low subscriber counts compared to their in-degree.

This metric was calculated using a Composite Z-Score Equation. The inverse of subscriber count s_i and in-degree d_i for each channel i were normalized using Z-Score normalization, and the “hidden influence” score H_i is the average of the two scores. In order to make results more interpretable, min-max normalization was used to scale all H_i to fit in the domain $[1, 100]$. The final reported score is represented as H_i^*

Mathematically, define $Z(x, \mu, \sigma)$ as a function that takes a value (subscriber count or degree) and normalizes it using the mean and standard deviation of each set. Then

$$Z(x, \mu, \sigma) = \frac{x - \mu}{\sigma}$$

For the sake of simplicity, if $\sigma = 0$, then $Z = 0$

Let $d_{i_{norm}}$ and $s_{i_{norm}}$ be the normalized Z-scores for degree and inverse subscriber count, respectively. Then:

$$d_{i_{norm}} = Z(d_i, \mu_{deg}, \sigma_{deg})$$

$$s_{i_{norm}} = -Z(s_i, \mu_{sub}, \sigma_{sub})$$

$$H_i = \frac{d_{i_{norm}} + s_{i_{norm}}}{2}$$

$$H_i^* = \frac{H_i - H_{min}}{H_{max} - H_{min}} \cdot 100$$

1.2 Community Insularity

The insularity of a community is the fraction of outgoing edges from nodes within the community that connect to other nodes within the community.

Let the set E_q represent all external edges (outgoing edges from a community q that connect outside of the community) and L_q represent all internal edges

(outgoing edges from a community q that connect to nodes in the community.). Then the insularity for a community q : $I(q)$ is defined as

$$I(q) = \frac{|L_q|}{|E_q| + |L_q|}$$

1.3 Degree Ratio

The degree ratio D for a a channel C is the ratio of in-degree d_{in_C} to out degree d_{out_C} .

$$D = \frac{d_{in_C}}{d_{out_C} + 1}$$

The +1 is to account for the $d_{out} = 0$ case

1.4 Centrality

This analysis relied on *betweenness centrality*, or how often a node lies on a shortest path between two other nodes. The topic is well covered in network science/graph theory literature, so I won't do a full treatment here. However, if you're interested, I encourage you to look at the foundational paper on betweenness centrality from 1977

1.5 Community Detection

The community detection algorithm used was the *Louvain method* with a resolution of $r = 1$.

In order to prevent hundreds or even thousands of communities from emerging, the algorithm was only applied to a subgraph G^* . G^* only has the nodes from the initial graph G that have 30,000 subscribers $s_C \geq 30,000$. Additionally, after the community detection completed, all communities of size < 10 were merged into one collective "Other".

For any nodes that did not have a community initially assigned to them (for not meeting the requirements), the node was assigned the community that the largest number of their neighbors shared. For instance, if a node C_0 was neighbors with 5 nodes from the "Core MCYT" community, and 3 from the "Modern MCYT" community, C_0 would be placed in the "Core MCYT" community.

I will not be doing a full treatment of the Louvain algorithm, but if you're interested, I encourage you to read the literature or Wikipedia on the subject

2 Cosine Similarity

2.1 Explanation

Cosine similarity is a technique to identify nodes that are similar to one another based on the similarity of their edges. Each node C_i is transformed into an N -dimensional vector: $(n_0, n_1, n_2, \dots, n_N)$ where each n_i represents the weight

given to a certain channel i . The weight is just how many times channel i was mentioned. For instance, if a channel 2 was mentioned more than a channel 1 which was mentioned more than a channel 0, which was not mentioned at all: then $n_2 > n_1 > n_0 = 0$.

By finding the cosine of the angle θ between two vectors, we can see how similar the outgoing mentions of two different channels are. $\cos\theta = 1$ implies a perfect match ($\theta = 0^\circ$), whereas $\cos\theta = 0$ implies no similarity whatsoever ($\theta = 90^\circ$).

2.2 Implementation

In order to achieve this, I constructed a "mention matrix" M , which is an $N \times N$ matrix such that each row r represents the vector for the channel C_r . Each column m represents every channel that has an outgoing mention to channel C_m .

Implementation Note: In the actual program implementation, a SciPy sparse matrix was used to avoid the computational issues that arise with a $78k \times 78k$ matrix.

To begin the explanation, note the definition of the cosine between two vectors $\mathbf{a}$ and $\mathbf{b}$:

$$\cos(\theta_{a,b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|}$$

Where $|\mathbf{x}|$ is the length of the vector, or more specifically: the *Euclidean length* or *L2 Norm*.

So it's plain to see that if the vectors were normalized such that only their *direction*, rather than their *magnitude* mattered, or in other words:

$$\mathbf{a}_{norm} = \frac{\mathbf{a}}{|\mathbf{a}|}$$

Then the equation for cosine similarity becomes

$$\cos(\theta_{a,b}) = \mathbf{a}_{norm} \cdot \mathbf{b}_{norm}$$

In other words: the cosine similarity is just the *dot product* of the two vectors.

Intuitively, this normalization means that channels with the same *fractions* of mentions going to specific creators point in the same direction. If C_1 mentions C_3 and C_4 2 and 4 times, respectively, then a channel C_2 that mentions C_3 and C_4 1 and 2 times, respectively, points in the same direction. Thus, the second step, after implementing the mention matrix, is to normalize each row with respect to the L2 norm. Then, the next step requires finding the row vector r corresponding to the target channel C_r : the channel to find others that are similar to it. In order to calculate the dot product between the r vector and all other vectors at once, we use matrix multiplication.

$$S = M \times r^T$$

Where S is an $N \times 1$ matrix where each row s_i is channel C_i 's similarity to channel C_r . Then simply sort the matrix, keeping track of which row corresponded to which channel, and the most similar channels can be returned. To prevent C_r matching itself, $s_r = -1$